

Manual & Roteiro de Testes — qsologbook.mjs

Diário de campo de radioamador no CouchDB

2026-10-01

Contents

Manual & Roteiro de Testes — qsologbook.mjs	1
1. Preparação do ambiente	1
2. Conhecendo o comando help e version	2
3. Conectividade: ping e setup	2
4. Sua estação: station	3
5. Gravando o primeiro QSO: add	3
6. Listando com list e filtros	4
7. Inspeccionando um QSO: show	5
8. Corrigindo um QSO: edit	5
9. Ciclo de vida do QSL: qsl	6
10. Removendo um QSO: delete	6
11. Rastreando mudanças: sync	7
12. A página de verificação no navegador	7
13. Limpeza e boas práticas	8
Apêndice A — Entendendo RST e S/R	8
Apêndice B — Entendendo _id e _rev	8
Apêndice C — Tabela de referência rápida	9
Encerramento	10
Apêndice D — Como regerar este PDF	10

Manual & Roteiro de Testes — qsologbook.mjs

Objetivo deste documento: servir ao mesmo tempo como **tutorial** (para quem nunca usou) e **roteiro de testes** (para validar cada funcionalidade). Siga na ordem. Cada seção explica *o que* o comando faz, *por que* existe e *como* verificar que funcionou.

Ambiente seguro: sempre trabalhe num banco descartável para não sujar o log real. Use COUCHDB_DB=qsologbook_briga (ou outro nome qualquer).

1. Preparação do ambiente

Objetivo: garantir que você está na pasta certa e que o banco usado é descartável.

```
cd ~/mysologbook
source ~/.bashrc
export COUCHDB_DB=qsologbook_briga
```

Por que isso importa:

- COUCHDB_DB define **qual banco** o qsologbook.mjs vai usar. Sem essa variável, o padrão é qsologbook — que pode conter seus contatos reais.
- source ~/.bashrc recarrega variáveis exportadas no arquivo de configuração do shell.

Verificação: rode echo \$COUCHDB_DB — deve imprimir qsologbook_briga.

Aviso de segurança: se em algum momento você colocou COUCHDB_PASSWORD em texto puro no ~/.bashrc ou neste transcript, **rotacione a senha no CouchDB** e atualize o arquivo. Variáveis de ambiente não são lugar seguro para segredos de longa duração.

2. Conhecendo o comando help e version

Objetivo: saber onde procurar a sintaxe antes de errar.

```
./qsologbook.mjs help
./qsologbook.mjs version
```

O que observar:

- help lista **todos os comandos** e, separadamente, **as flags de cada comando**. Isso é essencial: --rig só existe em station; --rst-sent só existe em add/edit.
- version mostra a versão do script — útil para relatar bugs.

Erros comuns que o help evita:

```
# ERRADO: faltou o comando (flag não é comando)
./qsologbook.mjs --callsign PU2YSJ --grid GG66
# erro: comando desconhecido: --callsign

# ERRADO: --rig não pertence ao add
./qsologbook.mjs add --callsign PU2YSJ --rig FT-991A
# erro: flag desconhecida para add: --rig
```

Regra de ouro: qsologbook <comando> [flags] — o comando vem sempre primeiro.

3. Conectividade: ping e setup

Objetivo: garantir que o CouchDB responde e que banco+índice existem.

```
./qsologbook.mjs ping
# -> CouchDB responde.

./qsologbook.mjs setup
# -> Banco e indice prontos.
```

Por que:

- ping testa a URL (COUCHDB_URL, padrão `http://127.0.0.1:5984`).
- setup cria o banco e o índice usado pelo list. É **idempotente**: pode rodar quantas vezes quiser.

Se ping falhar: verifique se o container está rodando (`podman ps | grep couchdb`) e se a URL/porta conferem.

4. Sua estação: station

Objetivo: gravar quem você é. Isso é metadado, não é um QSO.

```
# Gravar (com flags)
./qsologbook.mjs station --callsign PU2YSJ --grid GG66 --name Ulisses --rig FT-9
# -> Estacao gravada.
# -> PU2YSJ em GG66 (Ulisses)
```

```
# Ler (sem flags)
./qsologbook.mjs station
# -> PU2YSJ em GG66 (Ulisses)
```

Flags aceitas: `--callsign`, `--grid`, `--name`, `--rig`, `--antenna`.

Conceitos:

- **Indicativo (`--callsign`):** sua licença de radioamador (ex.: PU2YSJ).
- **Grid locator (`--grid`):** código Maidenhead de 4, 6 ou 8 caracteres (ex.: GG66 = São Paulo). Use o **seu** grid real.
- **Rig:** o rádio (ex.: FT-991A).

Teste de leitura/escrita: rode `station` sem flags antes e depois de gravar — a saída deve mudar.

A estação é gravada uma única vez e reutilizada. Não confunda com `add`, que cria um QSO novo.

5. Gravando o primeiro QSO: add

Objetivo: registrar um contato.

```
./qsologbook.mjs add --callsign PY2SSH --band 20m --mode CW \
--rst-sent 599 --rst-rcvd 599 --notes "primeira fina"
```

Saída esperada:

QSO gravado.

```
_id:      qso_2026-09-30_PY2SSH
_rev:     1-e04ff7cae50211b2639d00ebc8e51ff5
data/hora: 2026-09-30T19:34:46Z
indicativo: PY2SSH
banda:     B20m
modo:      CW
rst:       599 / 599
```

grid: -
qsl: pendente
dia: 2026-09-30

Atenção ao _id: o nome leva a data de **hoje** (UTC), não a do contato. No dia 30 ele sai qso_2026-09-30_PY2SSH; amanhã, qso_2026-10-01_PY2SSH. Todos os comandos daqui pra frente usam <id> — **copie o _id da saída**, em vez de escrever no lugar, ou os comandos vão reclamar que o documento não existe.

Flags obrigatórias: --callsign, --band, --mode.

Flags opcionais: --grid, --rst-sent, --rst-rcvd, --name, --qth, --dxcc, --country, --notes, --qsl, --timestamp.

O que entender:

Campo	Significado
id	Chave única. Formato: qso<dia>_<indicativo>. Se repetir, ganha sufixo -2, -3...
_rev	Revisão do CouchDB. Começa em 1-.... Muda a cada edição.
data/hora	UTC (Z no fim). Sem --timestamp, usa o agora .
banda	Internamente B20m; exibido como 20m.
rst	Formato enviado / recebido (o s/r do list).
qsl	Status padrão pendente.

Teste de unicidade: rode o mesmo add duas vezes. O segundo ganha _id com sufixo -2.

Teste de timestamp: adicione --timestamp 2026-01-15T10:00:00Z num add e confira que data/hora reflete o valor passado.

6. Listando com list e filtros

Objetivo: ver todos os QSOs e aprender a filtrar.

```
./qsologbook.mjs list
```

Saída típica:

```
_id | quando | indicativo | banda | modo | rst s/r | qsl
-----+-----+-----+-----+-----+-----+
qso_2026-09-30_PY2SSH-2 | 2026-09-30T19:38:27Z | PY2SSH | 20m | CW | 599/599 |
qso_2026-09-30_PY2SSH | 2026-09-30T19:34:46Z | PY2SSH | 20m | CW | 599/599 |
exibidos: 2
```

Ordem: mais recente primeiro.

Filtros disponíveis:

Filtro	Exemplo	O que faz
--callsign	--callsign PY2SSH	Só QSOs com esse indicativo
--band	--band 20m	Só dessa banda
--mode	--mode CW	Só desse modo
--qsl	--qsl confirmado	Só com esse status
--year	--year 2026	Só desse ano
--limit	--limit 5	No máximo N linhas

Roteiro de teste de filtros:

```
./qsologbook.mjs list --callsign PY2SSH
./qsologbook.mjs list --band 15m
./qsologbook.mjs list --mode CW
./qsologbook.mjs list --qsl confirmado
./qsologbook.mjs list --year 2026
./qsologbook.mjs list --limit 1
```

Cada um deve reduzir a lista. Combine filtros (--band 15m --qsl confirmado) para ver a interseção.

7. Inspeccionando um QSO: show

Objetivo: ver todos os campos de um QSO específico (inclusive os que o list não mostra).

```
./qsologbook.mjs show qso_2026-09-30_PY2SSH
```

Por que usar: o list mostra só colunas resumidas. O show revela _rev, grid, notes, dia, e tudo mais.

Teste: compare a saída de show antes e depois de um edit — veja o _rev mudar.

8. Corrigindo um QSO: edit

Objetivo: consertar um campo errado **sem recriar** o documento.

```
./qsologbook.mjs edit qso_2026-09-30_PY2SSH --band 15m --notes "corrigido, 15m"
```

Saída:

QSO atualizado.

```
_id:      qso_2026-09-30_PY2SSH
_rev:     2-404799ad360a8b29c2ddb86a4feaa32d
data/hora: 2026-09-30T19:34:46Z <-- preservado!
banda:    B15m
...
```

Conceitos-chave:

- **Só o que for citado muda.** Todo o resto permanece.
- **data/hora original é preservado** — o contato aconteceu naquele momento, a edição é só correção de digitação.

- **_rev incrementa** (1-... → 2-...). Isso é o controle de concorrência do CouchDB: se dois dispositivos editarem o mesmo doc, um conflito aparece.

Flags do edit: as mesmas do add. Pode corrigir banda, modo, RST, notes, etc.

Roteiro de teste:

1. Veja o estado atual

```
./qsologbook.mjs show qso_2026-09-30_PY2SSH
```

2. Corrija a banda

```
./qsologbook.mjs edit qso_2026-09-30_PY2SSH --band 15m --notes "corrigido, 15m"
```

3. Confirme

```
./qsologbook.mjs show qso_2026-09-30_PY2SSH
```

```
./qsologbook.mjs list --band 15m
```

Teste de preservação: anote o data/hora antes e depois do edit — deve ser idêntico.

9. Ciclo de vida do QSL: qsl

Objetivo: controlar a confirmação do contato (cartão QSL, LoTW, eQSL).

```
./qsologbook.mjs qsl qso_2026-09-30_PY2SSH enviado
```

-> QSL atualizado. qsl: enviado

```
./qsologbook.mjs qsl qso_2026-09-30_PY2SSH confirmado
```

-> QSL atualizado. qsl: confirmado

Estados possíveis: pendente, enviado, confirmado, recusado.

Uso no list:

```
./qsologbook.mjs list --qsl pendente
```

```
./qsologbook.mjs list --qsl confirmado
```

Por que existe: no radioamadorismo, um contato só é “oficial” quando confirmado por ambos os lados. O status QSL rastreia esse ciclo.

10. Removendo um QSO: delete

Objetivo: apagar um documento do banco.

```
./qsologbook.mjs delete qso_2026-09-30_PY2SSH-2
```

-> QSO qso_2026-09-30_PY2SSH-2 apagado.

Verificação:

```
./qsologbook.mjs list # não deve mais aparecer
```

```
./qsologbook.mjs show qso_2026-09-30_PY2SSH-2 # -> erro: não achei o documento
```

Cuidado: no CouchDB, o delete cria um **tombstone** — o documento some das consultas normais, mas o histórico fica no `_changes` (veja sync). Não é um “apaga para sempre”.

11. Rastreando mudanças: sync

Objetivo: ver o feed `_changes` do CouchDB desde o início.

```
./qsologbook.mjs sync
```

Saída típica:

```
seq | _id | op | quando | indicativo | banda | qsl
-----+-----+-----+-----+-----+-----+-----
-----+-----+-----+-----+-----+-----+-----
1-... | _design/qsologbook | atualizado | | | | |
2-... | station | atualizado | | | | |
6-... | qso_2026-09-30_PY2SSH | atualizado | 2026-09-30T19:34:46Z | PY2SSH | 15m
8-... | qso_2026-09-30_PY2SSH-2 | removido | | | | |
last_seq: 8-...
```

O que observar:

- **seq:** número sequencial do CouchDB. Cada mudança tem um.
- **op:** atualizado (criação ou edição) ou removido (delete/tombstone).
- **_design/qsologbook:** documento de design (índice) criado pelo setup.
- **last_seq:** última sequência — use para replicação incremental.

Por que existe: sync é a base para replicação entre dispositivos. Você pode pedir “o que mudou desde a seq X” e sincronizar só o delta.

Teste: rode add, depois sync, depois delete, depois sync de novo. Veja a sequência crescer e o op mudar.

12. A página de verificação no navegador

O que essa página é: `web/index.html` **não** é a interface do diário. É uma página que roda a camada de dados pura no navegador, contra os módulos compilados em `output/`, e mostra o resultado de cada asserção.

O que ela não faz: ela não fala com o CouchDB. Não há leitura de banco nenhuma nesse código. Um add feito pela CLI **não** aparece ali, porque não existe de onde a página buscar.

Em um terminal:

```
npm run serve
```

Abra `http://127.0.0.1:8080/web/`.

O que aparece é um resumo no topo e a lista de asserções com `pass` ou `fail`. Para validar a página sem olhar:

```
npm run check:web
```

Por que ela existe: o mesmo código de domínio roda no navegador e no Node. A página pega bugs que a suíte do Spago não pega, porque roda no motor em que a interface web vai rodar. Ela é o passo anterior a uma interface de verdade, que ainda não existe.

13. Limpeza e boas práticas

Ao terminar os testes:

Opção 1: apagar QSOs um a um

```
./qsologbook.mjs list  
./qsologbook.mjs delete <id>
```

Opção 2: apagar o banco inteiro (via curl)

```
curl -X DELETE http://127.0.0.1:5984/qsologbook_briga -u "admin:$COUCHDB_PASSWORD"
```

Boas práticas:

- **Nunca teste no banco real.** Sempre export `COUCHDB_DB=qsologbook_briga` (ou outro nome).
 - **Não coloque senha em texto puro** no `~/.bashrc` nem em transcripts.
 - **Faça backup** do banco real periodicamente (replicação CouchDB ou `curl` no `_all_docs`).
 - **Use `--timestamp`** para lançar contatos antigos retroativamente.
-

Apêndice A — Entendendo RST e S/R

RST = Readability + Strength + Tone.

Letra	Faixa	Significado
R	1-5	Legibilidade (1 = ilegível, 5 = perfeito)
S	1-9	Força do sinal (1 = muito fraco, 9 = muito forte)
T	1-9	Tom (só em CW; 1 = áspero, 9 = puro)

599 = R=5, S=9, T=9 → sinal perfeito. É o relatório clássico em CW.

rst s/r no list: s = enviado (o que **você** reportou), r = recebido (o que **ele** reportou). 599/599 = contato perfeito nos dois sentidos.

Em fonia (SSB/FM): usa-se só **RS** (2 dígitos, ex.: 59), porque tom não se aplica.

Apêndice B — Entendendo `_id` e `_rev`

`_id` — chave única do documento no CouchDB.

- Formato do `qsologbook`: `qso_<YYYY-MM-DD>_<INDICATIVO>`.
- Se já existe, ganha sufixo -2, -3... (evita sobrescrever contatos do mesmo dia com o mesmo indicativo).
- **Guarde esse nome** — é a chave para `show`, `edit`, `qsl`, `delete`.

`_rev` — número de revisão.

- Começa em 1-`<hash>`.
- **Muda a cada edição** (2-`<hash>`, 3-`<hash>`...).

- É o controle de concorrência do CouchDB: se dois dispositivos editarem o mesmo doc, um conflito aparece.
- **Nunca edite o `_rev` manualmente.**

Regra prática: `_id` identifica o *quê*; `_rev` identifica *qual versão*.

Apêndice C — Tabela de referência rápida

Comandos

Comando	Função	Flags principais
ping	Testa conexão	—
setup	Cria banco+índice	—
station	Lê/grava estação	--callsign --grid --name --rig --antenna
add	Cria QSO	--callsign --band --mode --rst-sent --rst-rcvd --notes --timestamp ...
list	Lista QSOs	--callsign --band --mode --qsl --year --limit
show	Vê 1 QSO	<id>
edit	Corrige QSO	<id> + flags do add
qsl	Muda status QSL	<id> <status>
delete	Apaga QSO	<id>
sync	Feed de mudanças	—
help	Mostra ajuda	—
version	Mostra versão	—

Status QSL

pendente → enviado → confirmado, ou recusado quando o outro lado nega

Variáveis de ambiente

Variável	Padrão	Uso
COUCHDB_URL	http://127.0.0.1:5984	Endereço do CouchDB
COUCHDB_DB	qsologbook	Nome do banco
COUCHDB_USER	(vazio)	Usuário (opcional)
COUCHDB_PASSWORD	(vazio)	Senha (opcional)

Fluxo típico de uso

ping → setup → station → add → list → show → edit → qsl → delete → sync

Erros comuns e solução

Erro	Causa	Solução
comando desconhecido: --callsign	Faltou o comando	Coloque add/station/etc. antes das flags
flag desconhecida para add: --rig	Flag de outro comando	Veja help; --rig é só do station
No such file or directory	Pasta errada	cd ~/myqsologbook
não consegui falar com o CouchDB	Serviço parado	podman ps \ grep couchdb
_rev conflito	Dois clientes editaram	Reabra com show e refaça o edit

Encerramento

Este roteiro cobre **todos os comandos** do qsologbook.mjs. Se você seguiu na ordem:

1. Entendeu o papel de cada variável de ambiente.
2. Criou e leu a estação.
3. Gravou, listou, inspecionou, editou, confirmou QSL e apagou um QSO.
4. Viu o feed de mudanças (sync) e a página de verificação do navegador.
5. Aprendeu RST, _id e _rev.

Próximos passos sugeridos:

- Testar filtros combinados no list.
- Criar QSOs com --timestamp retroativo e conferir no list --year.
- Rodar sync após cada operação para ver a sequência crescer.
- Abrir a página de verificação do navegador e passar npm run check:web.

Apêndice D — Como regerar este PDF

O comando é:

```
npm run manual
```

Que é o mesmo que:

```
pandoc manual-qsologbook.md -o manual-qsologbook.pdf \
  --pdf-engine=xelatex \
  -V geometry:margin=2.5cm \
  -V mainfont="DejaVu Sans" -V monofont="DejaVu Sans Mono" \
  -V papersize=a4 \
  --toc --toc-depth=2 \
  -V colorlinks=true -V linkcolor=blue
```

Duas coisas que quebram o build e que já custaram tempo:

papersize=a4 é obrigatório. Sem essa flag o PDF sai em tamanho carta, que não é o papel que se usa aqui.

A chave lang: não pode estar no frontmatter. O pandoc transforma pt-BR em babel-brazilian, e um TeX Live sem texlive-lang-portuguese falha o build inteiro com Unknown option 'brazilian'. Se você instalar esse pacote, pode voltar a colocar a chave e recuperar a hifenização em português:

```
sudo apt install texlive-lang-portuguese
```

Os emoji também saem de brinde: o DejaVu Sans não tem nenhum, e o que não existe na fonte vira caixa vazia no papel.